



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА



Имплементација и анализа Тендерминт консензус алгоритма

*Аутор: Алекса Наглић Е2 127/2025
Нови Сад, Јул 2025.*

Садржај

1. Теоријски увод.....	3
2. Модел система и проблем консензуса.....	3
3. Тендерминт алгоритама.....	5
4. Имплементација.....	6
4.1. Циљ и шта је имплементирано.....	6
4.2. Детаљан опис имплементације.....	6
Типови порука и ентитета.....	6
Структура ConsensusState.....	7
Извршавање и мрежни слој.....	7
Симулација грешке.....	8
4.3. Даља проширења.....	8
5. Закључак.....	9

1. Теоријски увод

У дистрибуираним системима, више чворова међусобно комуницирају путем мреже како би остварили заједнички циљ. Иако овакви системи нуде бројне предности, попут високе доступности, скалабилности и отпорности на отказе, они доносе и низ великих изазова. Највећи проблем представља чињеница да мрежа може бити непоуздана, поруке се могу изгубити или каснити, а сами чворови могу престати са радом или се понашати малициозно. Како би систем као целина наставио исправно да функционише, неопходно је да се исправни чворови усагласе око заједничког стања. Овакав процес се назива проблем консензуса.

2. Модел система и проблем консензуса

Консензус алгоритам **мора** да задовољи три основна својства:

1. **Слагање** (*Agreement*): Ниједна два исправна чвора не смеју донети различите одлуке.
2. **Валидност** (*Validity*): Одлучена вредност мора бити валидирана на основу предефинисаних правила.
3. **Завршетак** (*Termination*): Сваки исправан чвор мора пре или касније донети неку одлуку.

Постоје различити модели система у зависности од врсте отказа које толеришу:

- **Crash-fault tolerance (CFT)**: Предпоставља да чворови могу само престати са радом, али увек шаљу исправне информације.
- **Byzantine-fault tolerance (BFT)**: Предпоставља да чворови могу отказати, али се могу понашати потпуно произвољно (или злонамерно).

Модели дистрибуираних система се разликују по питању мрежне синхронизације:

- **Синхрони**
- **Асинхрони**
- **Делимично синхрони**

Тендерминт предпоставља да је модел система **делимично синхрон BFT**. У овом моделу предпоставља се да се мрежа може понашати потпуно асинхроно (поруке могу касинити бесконачно дуго) до одређеног непознатог тренутка које називамо **Глобално време стабилизације** (*Global Stabilization Time - GST*).

Код дистрибуираних система, **Безбедност** (*Safety*) и **Живост** (*Liveness*) су најбитније особине исправности и консензуса. Математички је доказано да у реалном свету је немогуће остварити оба у потпуности (*FLP Impossibility Theorem*) [1]. Како би се испунили ови услови морају да се уведу одређене предпоставке како систем функционише или да се прихвате одређени *trade-off*-и. Тендерминт их решава на следећи начин:

- **Безбедност**: Гарантује да неће доћи до рачвања мреже тако што уводу дво-фазно гласање, где су потребне две трећине гласова. Ово постиже тако што протокол не напредује даље све док се две трећине не сложи око валидне вредности (жртвује живост)
- **Живост**: Предпоставком да је систем делимично синхрон ограђује се тајмерима који се повећавају сваком рундом, што гарантује да ће сви исправни чворови у коначном тренутку успети да добију и пошаљу своје гласове

Математички је доказано да овакав модел захтева $n = 3f + 1$ чворова, где систем толерише максимално f византијских чворова [2].

3. Тендерминт алгоритам

Тендерминт је *BFT* консензус алгоритам који омогућава безбедну и конзистентну репликацију машине стања (*State Machine Replication - SMR*). Заснован је на концепту гласања у више фаза.

Алгоритам се извршава секвенцијално, генеришући блокове на одређеним **висинама** (*Heights*). Да би се донела одлука за једну висину, процес се може одвијати у више **рунди** (*Rounds*). Свака рунда се састоји од три главна корака (*Steps*):

1. **Propose**: Један чвор (*Proposer*) предлаже вредност (блок) и емитује **PROPOSAL** поруку.
2. **Prevote**: Сви чворови примају предлог, валидирају га и емитују **PREVOTE** поруку. Чворови могу гласати за **nil** уколико сматрају да предлог није валидан или предлог није стигао на време.
3. **Precommit**: Када чвор скупии више од две трећине **PREVOTE** гласова за исту вредност, он се **закључава** на ту вредност и емитује **PRECOMMIT** поруку. Закључавање гарантује безбедност јер чвор не може гласати за другу вредност у наредним рундама (осим ако не добије доказ о новијем кворуму).

Када чвор прикупи кворум **PRECOMMIT** порука, он доноси коначну одлуку (*Commit*), инкрементира висину и прелази на процесирање следећег блока. Уколико кворум није достигнут због мрежних кашњења или пада лидера, долази до истека тајмера и чворови прелазе у следећу рунду исте висине са новим лидером.

4. Имплементација

4.1. Циљ и шта је имплементирано

Циљ практичног дела пројекта био је развој симулатора дистрибуираних чворова који међусобно комуницирају и извршавају Тендерминт консензус алгоритам у складу са академском спецификацијом. Имплементирано је следеће:

- Конкурентно извршавање чворова као независних ентитета.
- Комуникација порукама (PROPOSAL, PREVOTE, PRECOMMIT) преко TCP канала
- Стање чвора које прати висину, рунде, корако, као и сигурносне варијабле (`lockedValue`, `lockedRound`, `validValue`, `validRound`).
- Решавање *timeout* догађаја који омогућавају алгоритму да пређе у наредну рунду или висину (`timeoutPropose`, `timeoutPrevote`, `timeoutPrecommit`, `timeoutCommit`).
- Симулација византијских и обичних грешака, са различитим понашањем чворова (`silent`, `double-vote`, `send-invalid`)

4.2. Детаљан опис имплементације

Симулатор је реализован у програмском језику ***Rust***, из коришћење асинхроног *runtime*-а ***Tokio***, који обезбеђује асинхронно и конкурентно извршавање мрежних операција и тајмера.

Код је организован у неколико кључних структура података (дефинисаних у `types.rs`) и централне машине стања (у `consensus.rs`):

Типови порука и ентитета

- `Value`: Представља сам блок података око ког се постиже консензус.
- `Proposal`: Порука којом лидер предлаже `Value`. Садржи идентификаторе за `height` и `round`, као и поље `valid_round` уколико лидер поново предлаже вредност из предходне рунде.

- **Vote:** Порука којом чворови гласају. Поље `vote_type` одређује да ли је у питању **Prevote** или **Precommit** фаза. Глас увек може бити везан за конкретан **Value** или може бити **None** (што одговара `nil` гласу у псеудокоду)

Структура `ConsensusState`

Језгро сваког чвора представља структура `ConsensusState`, која прати тренутни напредак и држи неопходне безбедносне вредности. Најбитнија поља су:

- Праћење фазе: `height` (тренутни блок), `round` (тренутна рунда покушаја) и `step` (**Propose**, **Prevote**, **Precommit**, **Commit**).
- Безбедносне променљиве: `locked_value` и `locked_round` онемогућавају чвору да гласа за различите вредности на истој висини уколико је већ послао **PRECOMMIT** поруку, чиме се спречава настанак дваразличита блока на истој висини.
- Променљиве за валидност: `valid_value` и `valid_round` чувају последњу вредност за коју је чвор прикупио кворум **PREVOTE** порука. Ово служи као *cache* да би се добра вредност пренела у наредну рунду ако кворум за **PRECOMMIT** изостане.
- Складиштење гласова: `votes` мапа (типа `HashMap<Round, RoundVote>`) у којој чвор агрегира пристигле **PREVOTE** и **PRECOMMIT** поруке по рундама како би знао када је достигао неопходан кворум.

Извршавање и мрежни слој

`ConsensusState` је искључиво синхрона, детерминистичка машина стања, док сви непредвидиви мрежни догађаји се дешавају асинхроно у `PerfectLink`-у. Спој између ова два слоја остварен је помоћу *MPSC* (*Multi-Producer Single-Consumer*) канала:

- **PerfectLink:** Комуникација са осталим чворовима је издвојена у посебан асинхрони *Tokio* таск. Ослушкивање долазних конекција (преко `TcpListener`-а) не блокира рад чвора. Када пакет стигне, `PerfectLink` га десеријализује, провери да ли је већ обрађен и шаље у `event_receiver` канал. Са друге стране, уколико чвор жели нешто да

пошаље, он то само гурне у `req_sender` канал који `PerfectLink` аутоматски покупи и шаље даље у мрежу.

- *Tokio select* петља: налази се у `main.rs` и функционише као главни *event loop*. Макро `tokio::select!` Конкурентно ослушкује и буди чвор искључиво када се деси један од два догађаја: (1) стигне порука из `event_receiver` канала или (2) истекне локални асинхрони `sleep_timer`. Када стигне порука са мреже, она се синхроно прослеђује функцијама `handle_proposal` или `handle_vote`.
- Тајмери: Уколико тајмер истекне, `select!` окида `handle_timeout` функцију која прелази у наредну фазу и евентуално враћа команду за слање. Имплементиран је додатни тајмер `timeoutCommit`, како би паузирао прелазак на нову висину брзих чворова.

Симулација грешке

Постоји могућност конфигурације грешака путем *enum*-а `Behaviour` (подешава се преко *environment* варијабле). `DoubleVote` симулира византијског актера који шаље два различита гласа у истој рунди, `SendInvalid` симулира чвор који предлаже малициозне блблокове, док `Silent` симулира такозвани *crash* отказ.

4.3. Даља проширења

Иако тренутни симулатор успешно реплицира логику консензуса и издржава малициозна понашања, он је у основи само фокусиран на *Consensus Layer*. Постоје ствари које недостају у односу на продукциони Тендерминт:

1. Апликативни слој (*SMR*)
2. Криптографски потписи: Тренутна имплементација верује пошиљаоцу поруке на основу `sender: NodeId` поља. У правом *BFT* систему, свака порука мора бити асиметрично криптографски потписана како малициозни чворови не би могли да лажирају идентитет других.
3. Динамички сет валидатора: Тренутни сет валидатора и њихова гласачка моћ је статична. Прави системи подржавају промене скупа валидатора (улазак и излазак из скупа)
4. *Metapool* и контрола креирања блокова: Чвор тренутно предлаже фиктивне блокове истог тренутка када постане лидер. У сварним

мрежама чворои скупљају трансакције у *Metpool*-у и уколико је он празан систем или чека, или шаље празан блок

5. Закључак

Тендерминт *BFT* пружа веома елегантан модел за решавање проблема византијског консензуса у делимично синхроним мрежама. Кроз имплементацију овог симулатора у језику *Rust* показано је како формална правила и сигуносни механизми спречавају појаву двоструких одлука. Истовремено, дизајн заснован на строгом праћењу тајмера показао се кључним за одржавање живости система, чак и када су одређени чворови спори или нефункционални.

Референце

- [1] Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1983). Impossibility of distributed consensus with one faulty process. *Proceedings of the 2nd ACM SIGACT-SIGMOD symposium on Principles of database systems*
- [2] Buchman, E., Kwon, J., & Milosevic, Z. (2018). The latest gossip on BFT consensus. arXiv preprint arXiv:1807.04938.